Rapport TP 3

AZDAD Samy - BENHADDYA Loqmann

Partie 1 - Formats de Donnees (Chapitre 3)

Tache 1 - Identifier les types JSON dans une reponse reelle

Requete GET vers https://jsonplaceholder.typicode.com/users/1 pour analyser les types JSON natifs (RFC 8259).



Illustration : Requete GET et reponse JSON de l'utilisateur

Identification des 6 types JSON vus en cours :

* string : Present. Exemple : 'name': 'Leanne Graham', 'email': 'Sincere@april.biz'.

* number : Present. Exemple : 'id': 1. (Remarque : lat et lng sont encodes en chaines de caracteres).

* boolean : Absent. Aucune valeur booleenne true ou false n'apparait dans ce profil.

* null : Absent. Aucun champ n'a la valeur nulle.

* object : Present. Exemple : 'address': { ... }, 'geo': { ... }, 'company': { ... }.

* array : Absent. Aucun tableau indexe [...] n'est present dans cette ressource unitaire.

Tache 2 - Convertir du JSON en XML

Transcription du JSON en document XML valide (fichier user.xml) respectant une racine unique, l'imbrication d'address et geo, et une modelisation rigoureuse :

```
â–† TÃ‚CHE 2 : Document XML valide transcrivant le JSON

$ cat user.xml

<?xml version="1.0" encoding="UTF-8"?>
<user id="1">
    <name>Leanne Graham</name>
    <username>Bret</username>
    <email>Sincere@april.biz</email>
    <address>
        <street>Kulas Light</street>
        <suite>Apt. 556</suite>
        <city>Gwenborough</city>
        <zipcode>92998-3874</zipcode>
        <geo>
            <lat>-37.3159</lat>
            <lng>81.1496</lng>
        </geo>
    </address>
    <phone>1-770-736-8031 x56442</phone>
    <website>hildegard.org</website>
    <company>
        <name>Romaguera-Crona</name>
        <catchPhrase>Multi-layered client-server neural-net</catchPhrase>
        <bs>harness real-time e-markets</bs>
    </company>
</user>
```

Illustration : Document XML complet transcrit

Question a consigner (Choix pour le champ id) :

Le champ id a ete modelise comme un ATTRIBUT de la racine (<user id='1'>). Selon les criteres vus en cours, les attributs sont reserves aux metadonnees techniques et identifiants uniques qui qualifient l'element sans faire partie de son contenu textuel metier propre. Les elements enfants (<name>, <email>, <address>) sont reserves aux donnees metier affichables, extensibles et composees.

Tache 3 - Negociation de contenu et codes d'erreur

Tests sur httpbin.org pour observer le comportement de la negociation de contenu :



Illustration : Statut HTTP 406 Not Acceptable

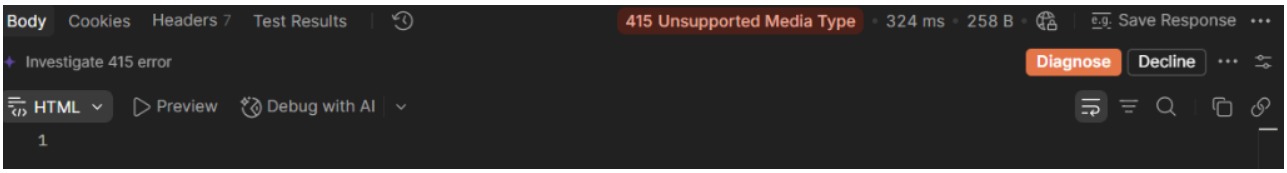


Illustration : Statut HTTP 415 Unsupported Media Type

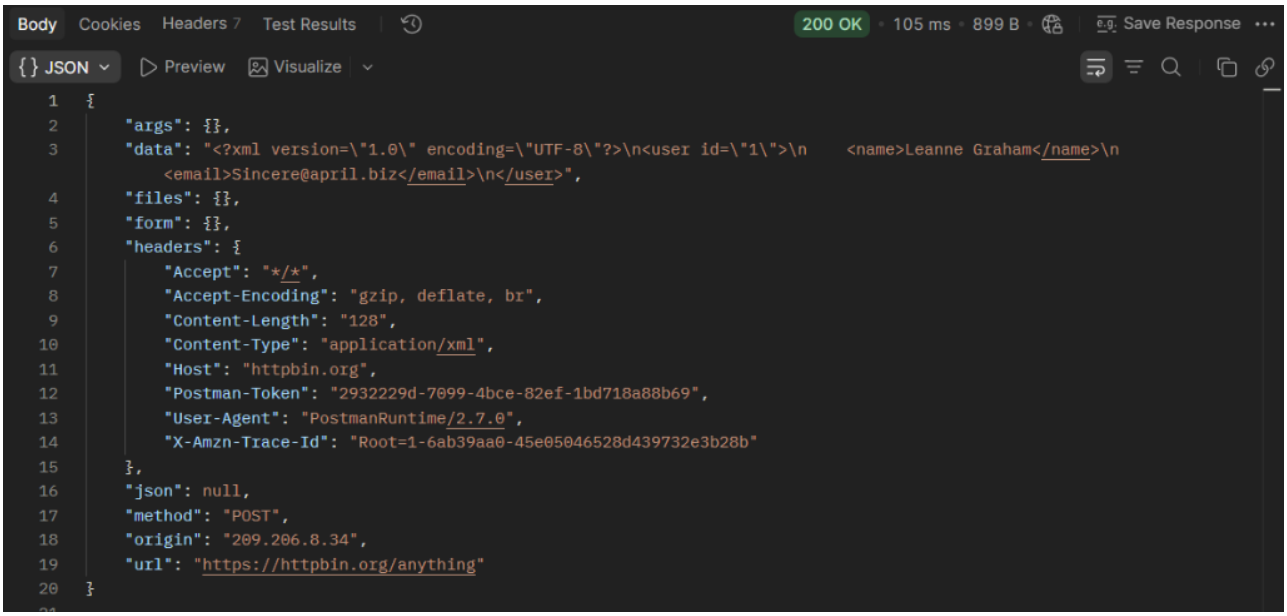


Illustration : POST sur /anything avec Content-Type: application/xml repercute dans les en-tetes

Question a consigner (Sens des codes 406 et 415) :

* Code 406 (Not Acceptable) : Le serveur renvoie cette erreur lorsque le format reclame par le client via l'en-tete Accept (ex: Accept: application/xml) ne peut pas etre produit par le serveur.

* Code 415 (Unsupported Media Type) : Le serveur renvoie cette erreur lorsqu'il refuse de traiter le corps de la requete parce que le format transmis via l'en-tete Content-Type (ex: XML) n'est pas pris en charge par l'endpoint cible.

Partie 2 - REST et SOAP (Chapitre 4)

Tache 4 - Concevoir des URLs RESTful

Conception de l'API REST d'une bibliotheque en ligne (ressources au pluriel, pas de verbe dans l'URI, query string pour le filtrage) :

Action metier	Methode	URL RESTful	Justification REST
Lister tous les livres	GET	/livres	Collection au pluriel, action de lecture
Recuperer le livre d'ID 42	GET	/livres/42	Identification directe par ID dans le chemin
Lister livres disponibles	GET	/livres?disponible=true	Filtrage par query string sans alterer l'URI
Emprunter livre 42	POST	/emprunts	Creation de la ressource emprunt (body: {livreId:42})
Livres de l'auteur 7	GET	/auteurs/7/livres	Sous-ressource imbriquee exprimant la relation
Mettre fin a l'emprunt 15	PATCH	/emprunts/15	Cloture de l'emprunt (ou DELETE pour suppression)

Tache 5 - Tester la non-idempotence de POST

Deux requetes POST consecutives executees avec le meme corps sur https://jsonplaceholder.typicode.com/posts :



Illustration : Deux envois POST successifs renvoyant id: 101

Question a consigner (Analyse et idempotence) :

Les deux reponses sont superficiellement identiques (id: 101) car JSONPlaceholder est un mock statique. Mais sur un serveur reel, reiterer POST entraine la creation successive de deux ressources distinctes (id=101 puis id=102). Cela demontre concretement que la methode POST n'est PAS idempotente : elle produit des effets de bord cumulatifs a chaque execution.

Tache 6 - Interroger un vrai service SOAP

Appel au service SOAP http://www.dneonline.com/calculator.asmx pour effectuer l'addition (4 + 7 = 11) :



Illustration : Reponse XML SOAP avec AddResult = 11

Questions a consigner :

* Identification : L'Envelope (<soap:Envelope>) delimite le conteneur du message et ses namespaces ; le Body (<soap:Body>) contient les donnees metier (<AddResponse> et <AddResult>11</AddResult>).

* Verbe HTTP : Toutes les requetes SOAP utilisent invariablement POST, quelle que soit l'action (Add, Subtract...). Contrairement a REST qui s'appuie sur la richesse des verbes HTTP (GET, POST, PUT, DELETE), SOAP utilise HTTP comme un simple tunnel aveugle (RPC tunneling).

* Localisation de l'operation : Le nom de l'operation (Add) apparait dans le corps XML et l'en-tete SOAPAction, jamais dans l'URL. Cela confirme que REST est oriente ressources alors que SOAP est oriente operations / procedures distantes (RPC).

Partie 3 - GraphQL et gRPC (Chapitre 5)

Tache 7 - Ecrire une requete GraphQL contre une vraie API

Execution d'une requete sur l'API Rick and Morty (personnage 1 + ses 51 episodes) :

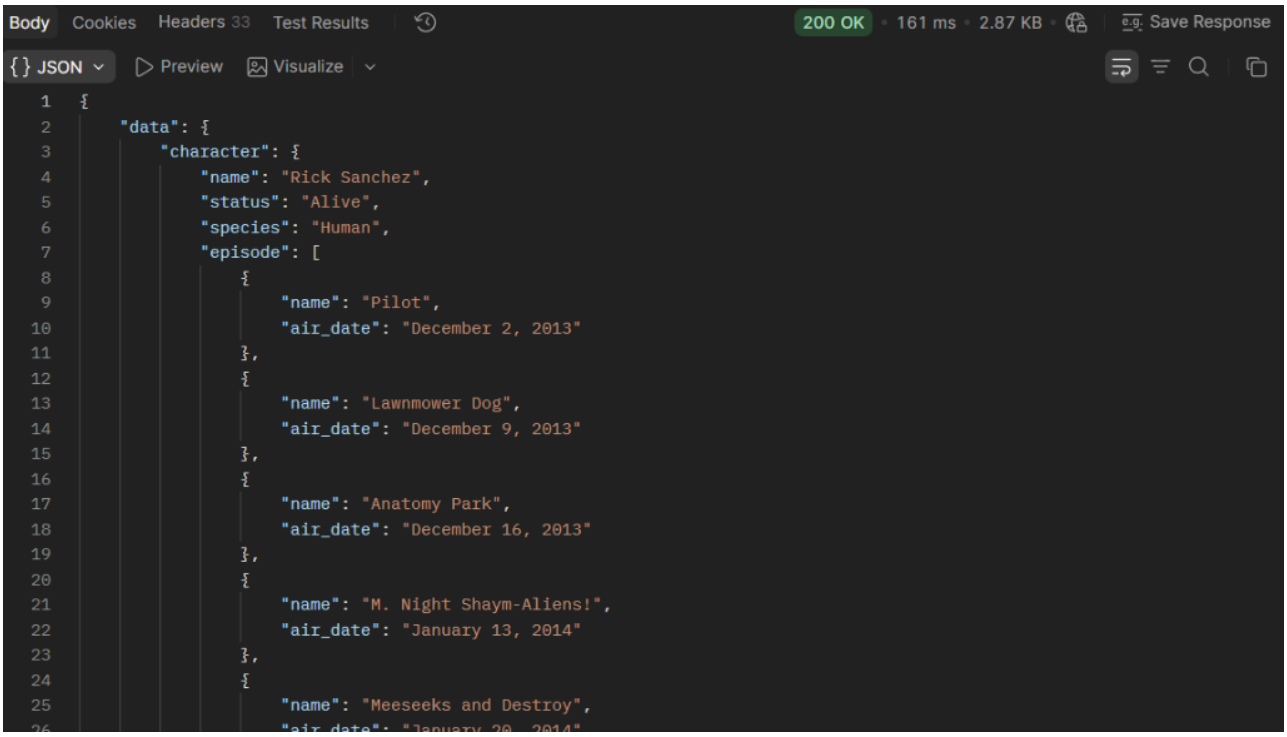


Illustration : Requete GraphQL combinant character et 51 episodes

Question a consigner (REST vs GraphQL et under-fetching) :

Avec une API REST classique (GET /character/1), l'appel ne renverrait que la liste des URLs des episodes. Pour obtenir le nom et la date de diffusion de chacun, le client devrait faire 51 requetes supplementaires, soit 52 requetes HTTP au total (probleme du N+1). L'under-fetching designe cette insuffisance de donnees dans un endpoint REST obligeant a multiplier les allers-retours reseau. GraphQL le resout en permettant d'extraire tout le graphe lie en un unique appel.

Tache 8 - Modifier la requete pour observer l'over-fetching

Execution de la meme requete allegee pour ne demander que le nom :



Illustration : Requete GraphQL ciblee : 49 octets seulement

Question a consigner (Comparaison de taille et over-fetching) :

La reponse detaillee (Tache 7) pese 3 537 octets alors que la requete ciblee (Tache 8) ne pese que 49 octets. En REST, l'endpoint renvoie systematiquement la totalite des 16 champs du personnage meme si le client n'en veut qu'un seul (over-fetching). GraphQL elimine ce gaspillage en laissant le client choisir exactement les champs dont il a besoin.

Tache 9 - Concevoir un schema GraphQL

Schema complet modelise dans schema.graphql pour la bibliotheque :

```
type Livre {
```

```
  id: ID!
  titre: String!
  isbn: String!
  disponible: Boolean!
  auteur: Auteur!
  emprunts(enCoursSeulement: Boolean = true): [Emprunt!]!
}
type Auteur {
  id: ID!
  nom: String!
  livres: [Livre!]!
}
type Emprunt {
  id: ID!
  dateEmprunt: String!
  dateRetourPrevue: String!
  enCours: Boolean!
  livre: Livre!
  utilisateur: Utilisateur!
}
type Utilisateur {
  id: ID!, nom: String!, email: String!
}
type Query {
  livre(id: ID!): Livre
  livres(disponible: Boolean): [Livre!]!
  auteur(id: ID!): Auteur
}
```

Justifications : Typage strict avec ID! et non-nullite (!), argument de filtrage enCoursSeulement sur la relation des emprunts pour eviter les transferts superflus, et relations bidirectionnelles navigables.

Tache 10 - Concevoir un service gRPC

Definition Protocol Buffers proto3 dans bibliotheque.proto :

```
syntax = "proto3";
package bibliotheque;

message LivreRequest {
  int32 id = 1;
}
```

```
message LivreResponse {
  int32 id = 1;
  string titre = 2;
  string auteur = 3;
  bool disponible = 4;
}
message StreamEmpruntsRequest {
  string categorie_filtre = 1;
}
message EmpruntNotification {
  int32 emprunt_id = 1;
  int32 livre_id = 2;
  string titre_livre = 3;
  string timestamp = 4;
}
service LivreService {
  rpc GetLivre (LivreRequest) returns (LivreResponse);
  rpc SuivreEmpruntsEnTempsReel (StreamEmpruntsRequest) returns (stream EmpruntNotification);
}
```

Justifications des modes de streaming :

* Methode GetLivre -> Mode Unaire (Unary RPC) : La consultation ponctuelle d'un livre correspond au paradigme classique 1 requete -> 1 reponse. C'est le mode le plus direct et le plus performant pour une lecture unitaire.

* Tableau de bord des emprunts en continu -> Mode Server Streaming : Le client ouvre une connexion initiale d'ecoute et le serveur maintient la connexion HTTP/2 ouverte pour pousser (push) les evenements au fil de l'eau (stream). Contrairement au mode unaire qui ferme la connexion apres une seule reponse, le server streaming etablit un flux persistant temps reel sans polling.

Synthese Comparative Globale

Critere	SOAP	REST	GraphQL	gRPC
Transport	HTTP (tunnel aveugle)	HTTP/1.1 ou 2	HTTP (POST)	HTTP/2 (binaire)
Format	XML (Enveloppe rigide)	JSON / XML	JSON personnalise	Protobuf (Binaire)
Paradigme	RPC (Procedures)	Ressources (CRUD)	Graphe declaratif	RPC Type rapide
Under-fetching	Selon methode	Frequent (N+1)	Resolu	Gere par messages
Over-fetching	Payloads rigides	Frequent	Resolu	Compact / Binaire